



《模式识别》

第十章 卷积神经网络及经典框架

马锦华

<https://cse.sysu.edu.cn/teacher/MaJinhua>

SUN YAT-SEN University



声明：该PPT只供非商业使用，也不可视为任何出版物。由于历史原因，许多图片尚没有标注出处，如果你知道图片的出处，欢迎告诉我们 majh8@mail.sysu.edu.cn.



课程目录（暂定）

❑	第一章	课程简介与预备知识	6学时
❑	第二章	特征提取与表示	6学时
❑	第三章	主成分分析	3学时
❑	第四章	归一化、判别分析、人脸识别	3学时
❑	第五章	EM算法与聚类	3学时
❑	第六章	贝叶斯决策理论	3学时
❑	第七章	线性分类器与感知机	3学时
❑	第八章	支持向量机	3学时
❑	第九章	神经网络、正则项和优化方法	3学时
❑	第十章	卷积神经网络及经典框架	6学时
❑	第十一章	循环神经网络	2学时
❑	第十二章	Transformer	2学时
❑	第十三章	自监督与半监督学习	2学时
❑	第十四章	开放世界模式识别	2学时

3

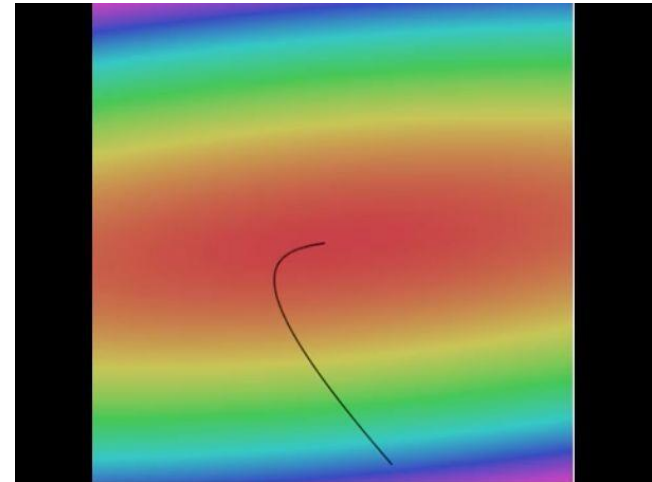
Where we are now...

- Learning network parameters through optimization



Landscape image is CC0 1.0 public domain

Walking man image is CC0 1.0 public domain



```
# Vanilla Gradient Descent
```

```
while True:
```

```
    weights_grad = evaluate_gradient(loss_fun, data, weights)
```

```
    weights += - step_size * weights_grad # perform parameter update
```



Where we are now...

- **Mini-batch SGD**

Loop:

1. **Sample** *a batch of data*
2. **Forward** *prop it through the graph (network), get loss*
3. **Backprop** *to calculate the gradients*
4. **Update** *the parameters using the gradient*



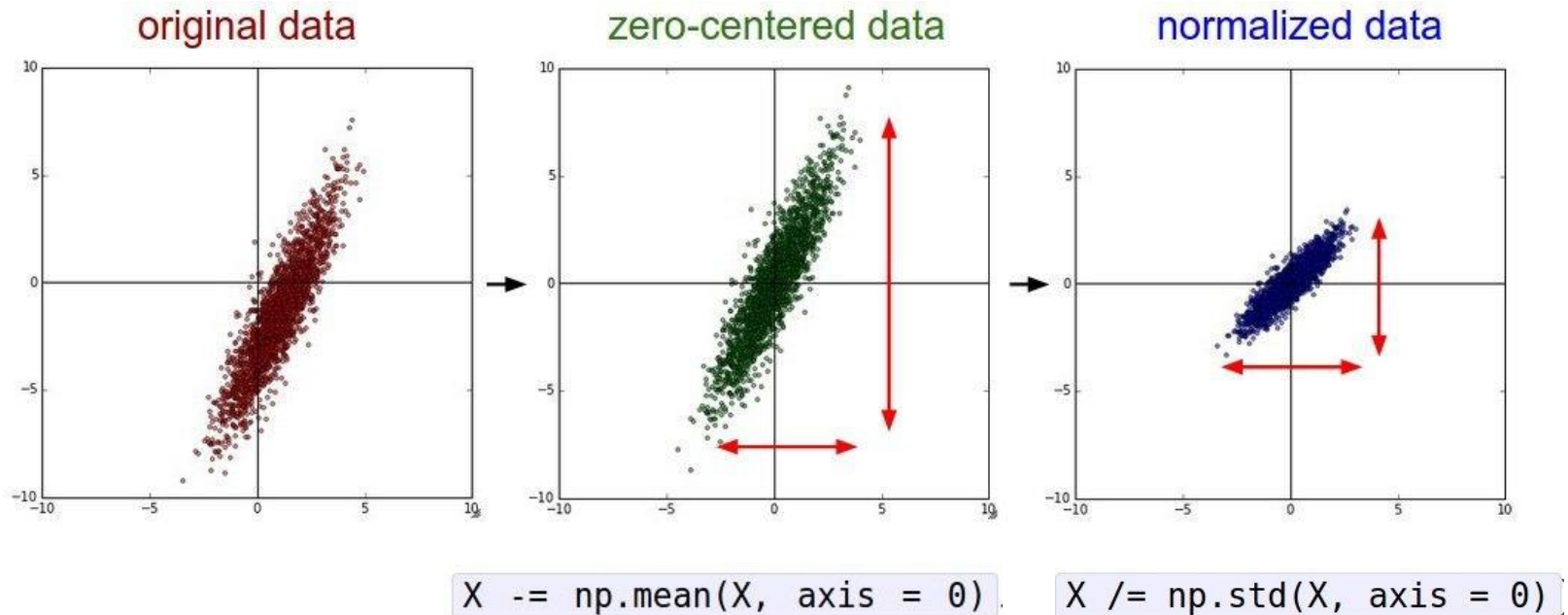
Today: Training Neural Networks

Overview

1. **One time set up:** *architecture, preprocessing, weight initialization, regularization, gradient checking*
2. **Training dynamics:** *babysitting the learning process, parameter updates, hyperparameter optimization*
3. **Evaluation:** *model ensembles, test-time augmentation, transfer learning*

Data Preprocessing

Data Preprocessing

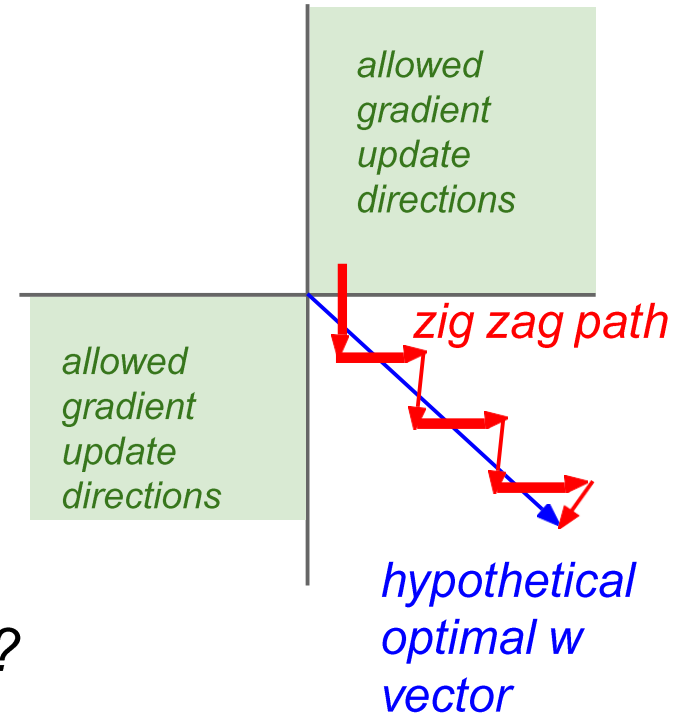


(Assume $X [N \times D]$ is data matrix, each example in a row)

Activation Functions

Remember: consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$



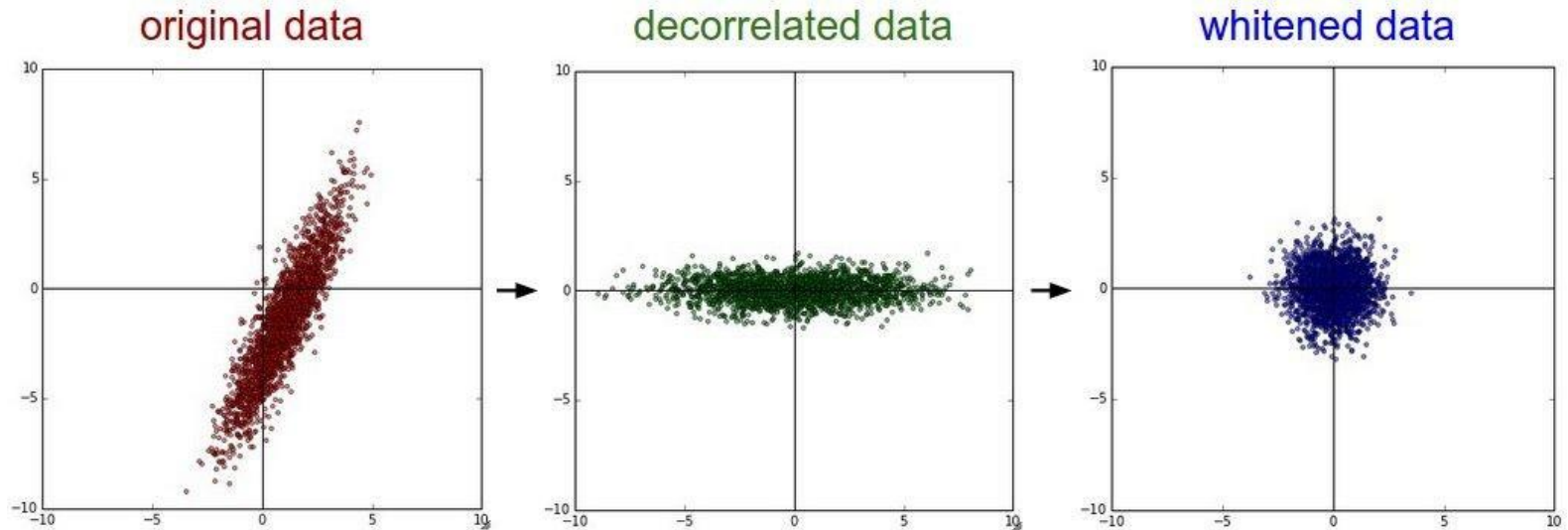
What can we say about the gradients on $\mathbf{w}$?

Always all positive or all negative :(

(this is also why you want zero-mean normalized data!)

Data Preprocessing

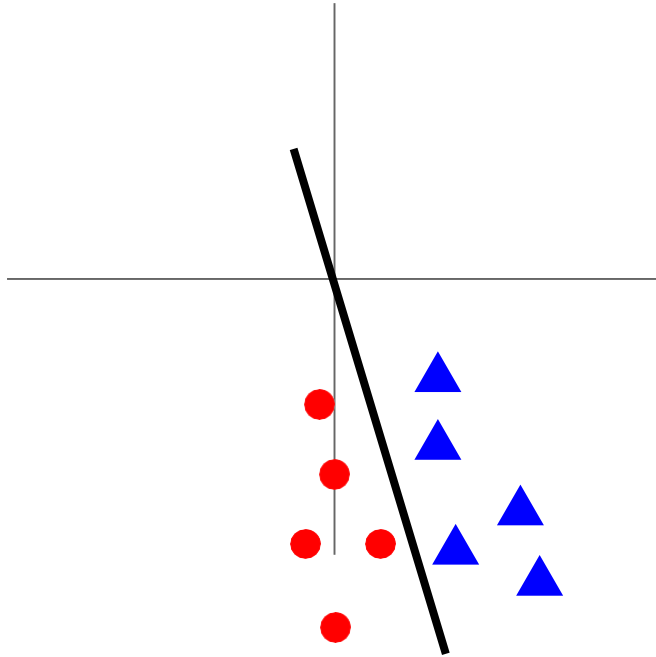
*In practice, you may also see **PCA** and **Whitening** of the data*



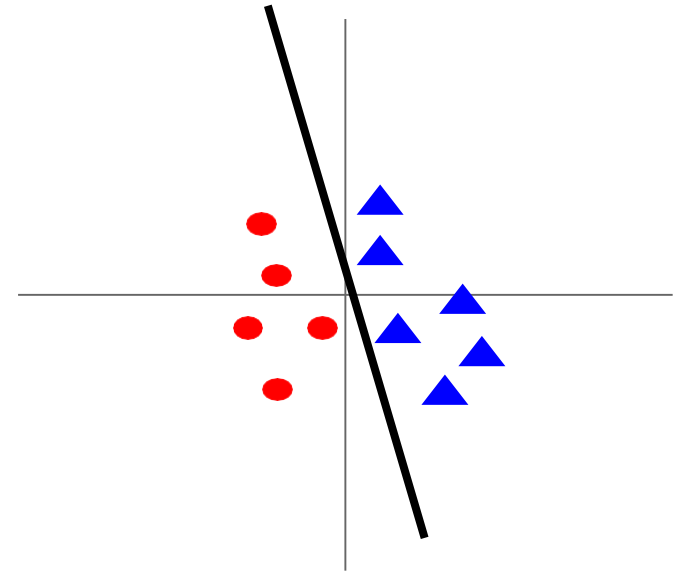
*(data has diagonal
covariance matrix)*

*(covariance matrix is
the identity matrix)*

Data Preprocessing



Before normalization:
*classification loss
very sensitive to changes in
weight matrix; hard to optimize*



After normalization: *less
sensitive to small changes
in weights; easier to
optimize*

Data Preprocessing

TLDR: In practice for Images: center only

e.g. consider CIFAR-10 example with [32,32,3] images

- *Subtract the mean image (e.g. AlexNet)*
- *(mean image = [32,32,3] array)*
- *Subtract per-channel mean (e.g. VGGNet)*
- *(mean along each channel = 3 numbers)*
- *Subtract per-channel mean and*

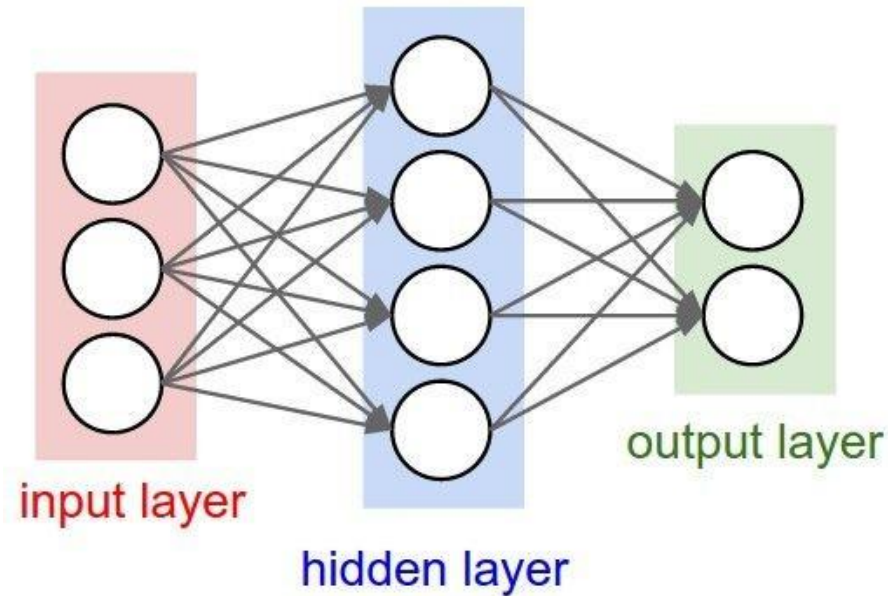
Divide by per-channel std (e.g. ResNet and beyond)
(mean along each channel = 3 numbers)

Not common to do PCA or whitening

Weight Initialization

Weight Initialization

- Q: what happens when $W=\text{constant init}$ is used?



Weight Initialization

- First idea: **Small random numbers**
(gaussian with zero mean and $1e-2$ standard deviation)

```
W = 0.01 * np.random.randn(Din, Dout)
```




Weight Initialization

- First idea: **Small random numbers**
(gaussian with zero mean and $1e-2$ standard deviation)

```
W = 0.01 * np.random.randn(Din, Dout)
```

Works ~okay for small networks, but problems with deeper networks.

Weight Initialization

Weight Initialization: Activation statistics

```
dims = [4096] * 7      Forward pass for a 6-layer  
hs = []               net with hidden size 4096  
x = np.random.randn(16, dims[0])  
for Din, Dout in zip(dims[:-1], dims[1:]):  
    W = 0.01 * np.random.randn(Din, Dout)  
    x = np.tanh(x.dot(W))  
    hs.append(x)
```

What will happen to the activations for the last layer?

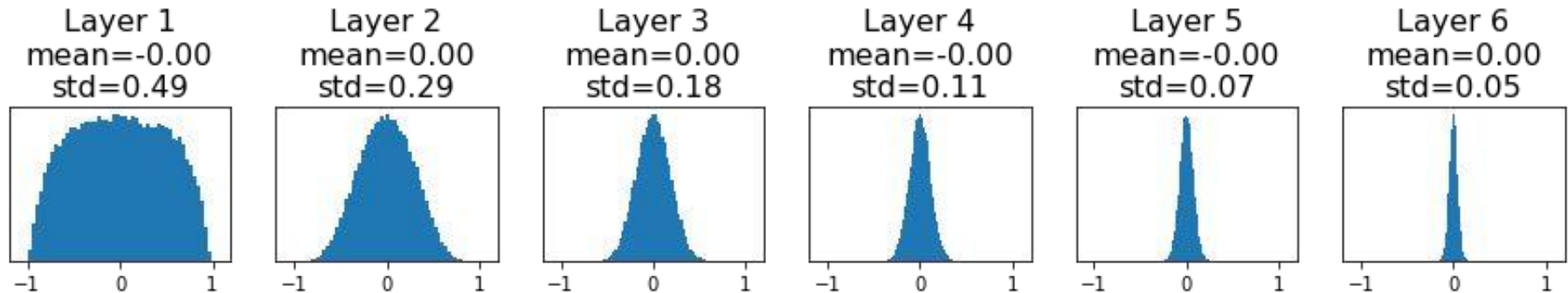
Weight Initialization

Weight Initialization: Activation statistics

```
dims = [4096] * 7      Forward pass for a 6-layer  
hs = []                net with hidden size 4096  
x = np.random.randn(16, dims[0])  
for Din, Dout in zip(dims[:-1], dims[1:]):  
    W = 0.01 * np.random.randn(Din, Dout)  
    x = np.tanh(x.dot(W))  
    hs.append(x)
```

All activations tend to zero for deeper network layers

Q: What do the gradients dL/dW look like?



Weight Initialization

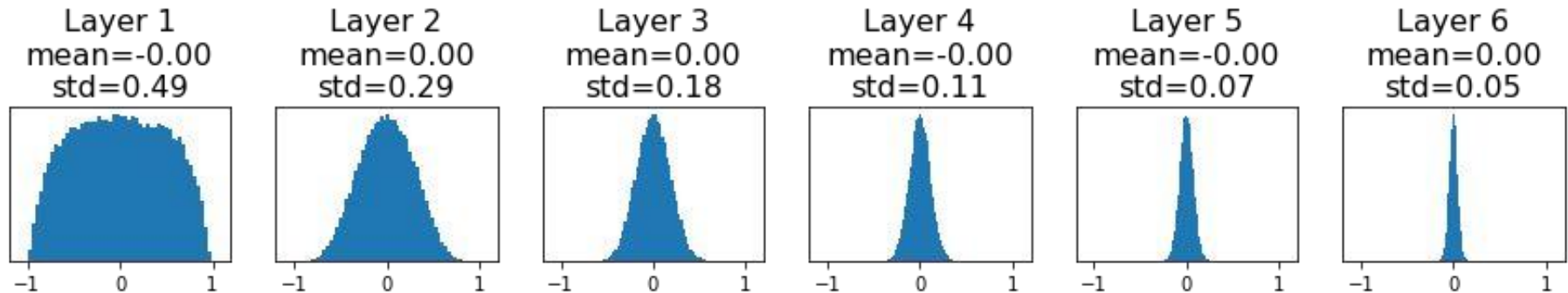
Weight Initialization: Activation statistics

```
dims = [4096] * 7      Forward pass for a 6-layer  
hs = []                net with hidden size 4096  
x = np.random.randn(16, dims[0])  
for Din, Dout in zip(dims[:-1], dims[1:]):  
    W = 0.01 * np.random.randn(Din, Dout)  
    x = np.tanh(x.dot(W))  
    hs.append(x)
```

All activations tend to zero for deeper network layers

Q: *What do the gradients dL/dW look like?*

A: *All zero, no learning =(*



Weight Initialization

Weight Initialization: Activation statistics

```
dims = [4096] * 7 Increase std of initial  
hs = [] weights from 0.01 to 0.05  
x = np.random.randn(16, dims[0])  
for Din, Dout in zip(dims[:-1], dims[1:]):  
    W = 0.05 * np.random.randn(Din, Dout)  
    x = np.tanh(x.dot(W))  
    hs.append(x)
```

What will happen to the activations for the last layer?

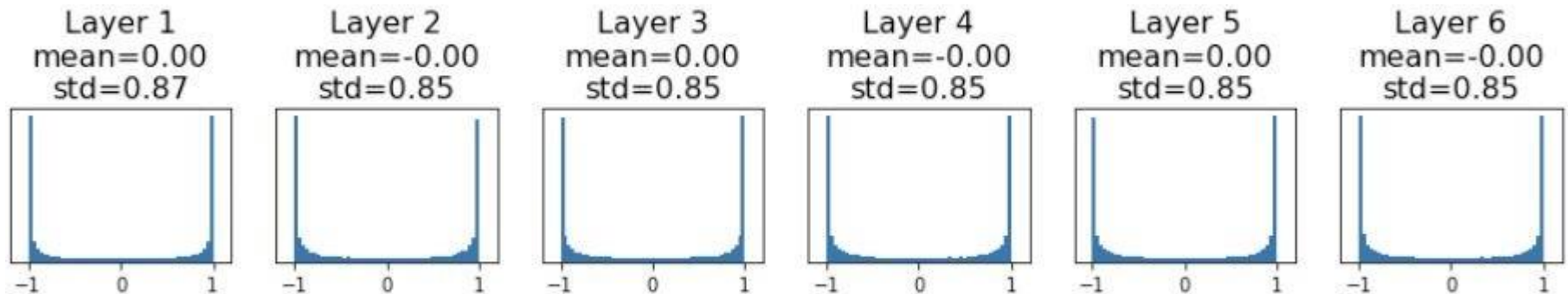
Weight Initialization

Weight Initialization: Activation statistics

```
dims = [4096] * 7 Increase std of initial  
hs = [] weights from 0.01 to 0.05  
x = np.random.randn(16, dims[0])  
for Din, Dout in zip(dims[:-1], dims[1:]):  
    W = 0.05 * np.random.randn(Din, Dout)  
    x = np.tanh(x.dot(W))  
    hs.append(x)
```

All activations saturate

*Q: What do the
gradients look like?*



Weight Initialization

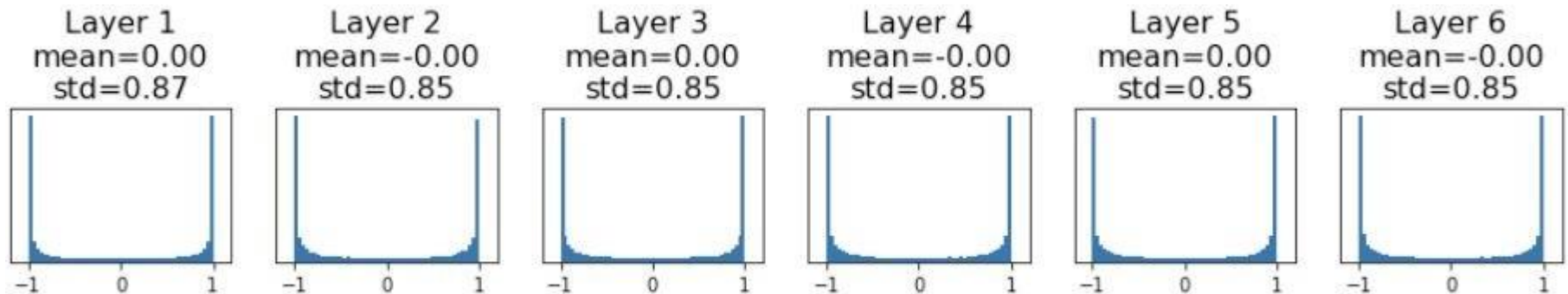
Weight Initialization: Activation statistics

```
dims = [4096] * 7 Increase std of initial  
hs = [] weights from 0.01 to 0.05  
x = np.random.randn(16, dims[0])  
for Din, Dout in zip(dims[:-1], dims[1:]):  
    W = 0.05 * np.random.randn(Din, Dout)  
    x = np.tanh(x.dot(W))  
    hs.append(x)
```

All activations saturate

Q: What do the gradients look like?

A: Local gradients all zero, no learning =(



Weight Initialization

Weight Initialization: “Xavier” Initialization

```
dims = [4096] * 7          “Xavier” initialization:
hs = []                    std = 1/sqrt(Din)
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

Glorot and Bengio, “Understanding the difficulty of training deep feedforward neural networks”, AISTAT 2010

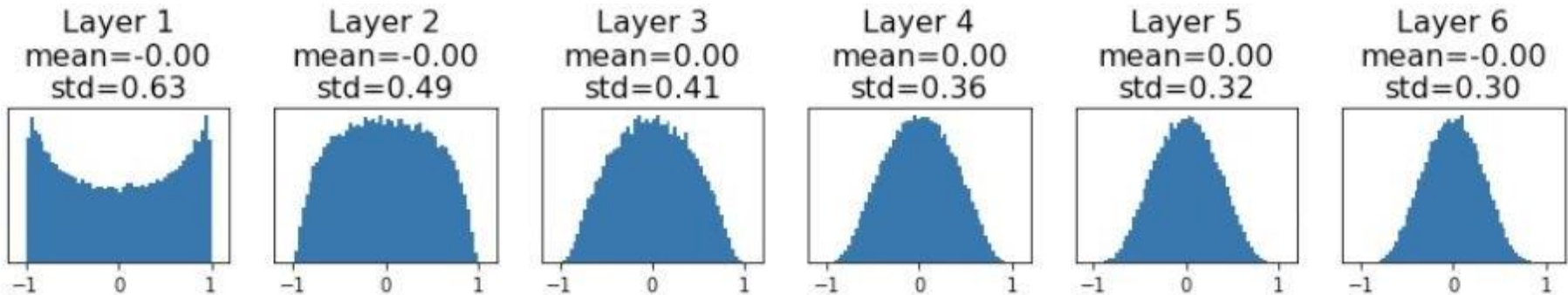
Weight Initialization

Weight Initialization: “Xavier” Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Xavier” initialization:
 $\text{std} = 1/\sqrt{D_{\text{in}}}$

“Just right”: Activations are nicely scaled for all layers!



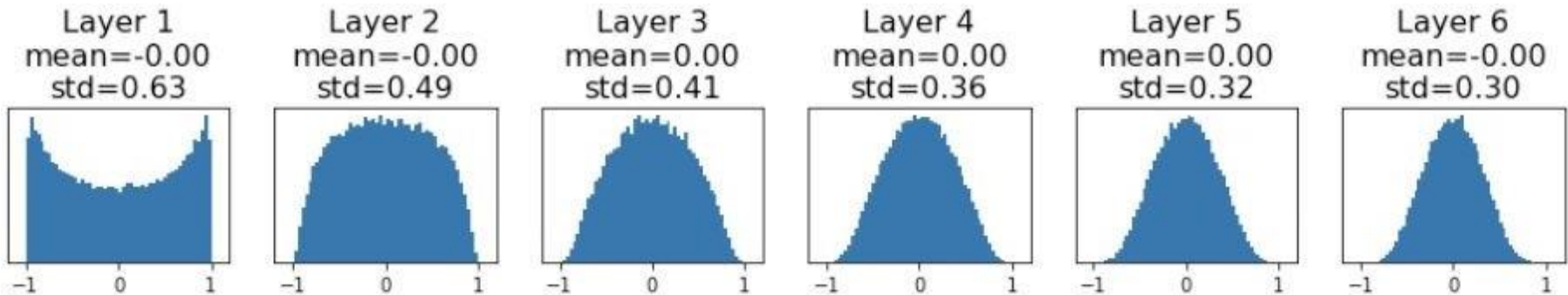
Weight Initialization

Weight Initialization: “Xavier” Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Xavier” initialization:
 $\text{std} = 1/\sqrt{D_{in}}$

“Just right”: Activations are nicely scaled for all layers!



For conv layers, D_{in} is $\text{filter_size}^2 * \text{input_channels}$

Weight Initialization

Weight Initialization: “Xavier” Initialization

```
dims = [4096] * 7          “Xavier” initialization:
hs = []                    std = 1/sqrt(Din)
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Just right”: Activations are nicely scaled for all layers!

For conv layers, D_{in} is $filter_size^2 * input_channels$

Let: $y = x_1 w_1 + x_2 w_2 + \dots + x_{D_{in}} w_{D_{in}}$

Weight Initialization

Weight Initialization: “Xavier” Initialization

```
dims = [4096] * 7          “Xavier” initialization:
hs = []                    std = 1/sqrt(Din)
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Just right”: Activations are nicely scaled for all layers!

For conv layers, D_{in} is $filter_size^2 * input_channels$

Let: $y = x_1 w_1 + x_2 w_2 + \dots + x_{D_{in}} w_{D_{in}}$

Assume: $Var(x_1) = Var(x_2) = \dots = Var(x_{D_{in}})$

Weight Initialization

Weight Initialization: “Xavier” Initialization

```
dims = [4096] * 7          “Xavier” initialization:
hs = []                    std = 1/sqrt(Din)
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Just right”: Activations are nicely scaled for all layers!

For conv layers, D_{in} is $filter_size^2 * input_channels$

Let: $y = x_1 w_1 + x_2 w_2 + \dots + x_{D_{in}} w_{D_{in}}$

Assume: $Var(x_1) = Var(x_2) = \dots = Var(x_{D_{in}})$

We want: $Var(y) = Var(x_i)$

Weight Initialization

Weight Initialization: “Xavier” Initialization

```
dims = [4096] * 7          “Xavier” initialization:
hs = []                    std = 1/sqrt(Din)
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Just right”: Activations are nicely scaled for all layers!

For conv layers, D_{in} is $filter_size^2 * input_channels$

Let: $y = x_1 w_1 + x_2 w_2 + \dots + x_{D_{in}} w_{D_{in}}$

Assume: $Var(x_1) = Var(x_2) = \dots = Var(x_{D_{in}})$

We want: $Var(y) = Var(x_i)$

$Var(y) = Var(x_1 w_1 + x_2 w_2 + \dots + x_{D_{in}} w_{D_{in}})$
[substituting value of y]

Weight Initialization

Weight Initialization: “Xavier” Initialization

```
dims = [4096] * 7          “Xavier” initialization:
hs = []                    std = 1/sqrt(Din)
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Just right”: Activations are nicely scaled for all layers!

For conv layers, D_{in} is $filter_size^2 * input_channels$

Let: $y = x_1 w_1 + x_2 w_2 + \dots + x_{D_{in}} w_{D_{in}}$

Assume: $Var(x_1) = Var(x_2) = \dots = Var(x_{D_{in}})$

We want: $Var(y) = Var(x_i)$

$$\begin{aligned} Var(y) &= Var(x_1 w_1 + x_2 w_2 + \dots + x_{D_{in}} w_{D_{in}}) \\ &= \textcolor{red}{D_{in} Var(x_i w_i)} \\ &[Assume all x_i, w_i are iid] \end{aligned}$$

Weight Initialization

Weight Initialization: “Xavier” Initialization

```
dims = [4096] * 7          “Xavier” initialization:
hs = []                    std = 1/sqrt(Din)
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Just right”: Activations are nicely scaled for all layers!

For conv layers, D_{in} is $filter_size^2 * input_channels$

Let: $y = x_1 w_1 + x_2 w_2 + \dots + x_{D_{in}} w_{D_{in}}$

Assume: $Var(x_1) = Var(x_2) = \dots = Var(x_{D_{in}})$

We want: $Var(y) = Var(x_i)$

$$\begin{aligned} Var(y) &= Var(x_1 w_1 + x_2 w_2 + \dots + x_{D_{in}} w_{D_{in}}) \\ &= D_{in} Var(x_i w_i) \\ &= D_{in} Var(x_i) Var(w_i) \\ &[Assume all x_i, w_i are zero mean] \end{aligned}$$

Weight Initialization

Weight Initialization: “Xavier” Initialization

```
dims = [4096] * 7          “Xavier” initialization:
hs = []                    std = 1/sqrt(Din)
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Just right”: Activations are nicely scaled for all layers!

For conv layers, D_{in} is $filter_size^2 * input_channels$

Let: $y = x_1 w_1 + x_2 w_2 + \dots + x_{D_{in}} w_{D_{in}}$

Assume: $Var(x_1) = Var(x_2) = \dots = Var(x_{D_{in}})$

We want: $Var(y) = Var(x_i)$

$Var(y) = Var(x_1 w_1 + x_2 w_2 + \dots + x_{D_{in}} w_{D_{in}})$

$= D_{in} Var(x_i w_i)$

$= D_{in} Var(x_i) Var(w_i)$

[Assume all x_i, w_i are zero mean]

So, $Var(y) = Var(x_i)$ only when $Var(w_i) = 1/D_{in}$

Weight Initialization

Weight Initialization: What about ReLU?

```
dims = [4096] * 7      Change from tanh to ReLU
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

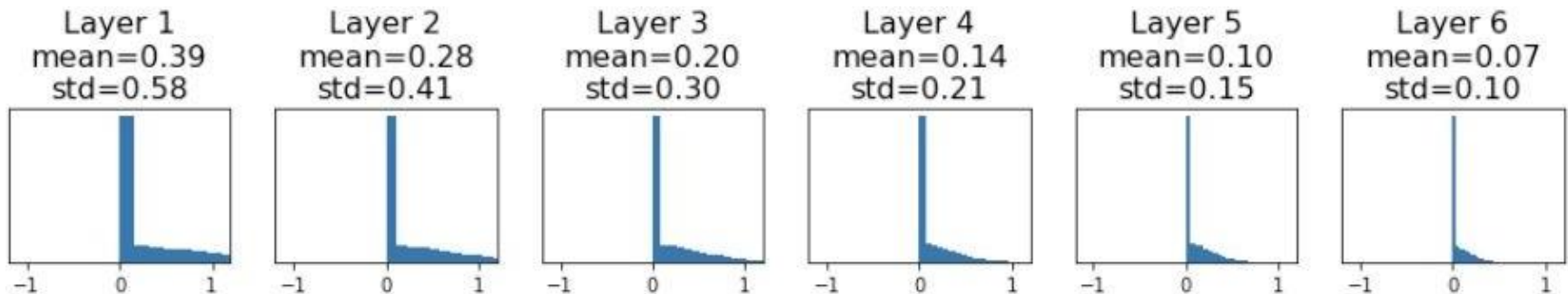
Weight Initialization

Weight Initialization: What about ReLU?

```
dims = [4096] * 7      Change from tanh to ReLU
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

Xavier assumes zero centered activation function

Activations collapse to zero again, no learning =(

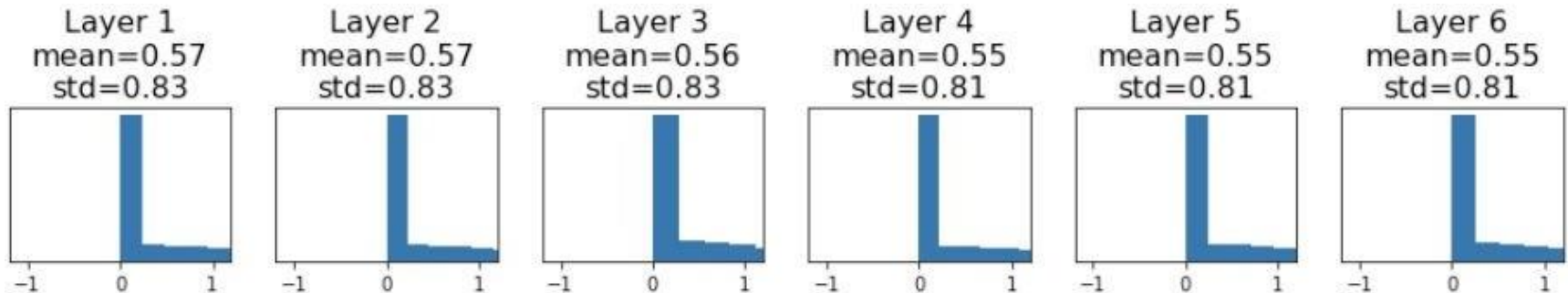


Weight Initialization

Weight Initialization: Kaiming / MSRA Initialization

```
dims = [4096] * 7 ReLU correction: std = sqrt(2 / Din)
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) * np.sqrt(2/Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

“Just right”: Activations are nicely scaled for all layers!



He et al, “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification”, ICCV 2015

Weight Initialization

Proper initialization is an active area of research...

Understanding the difficulty of training deep feedforward neural networks by Glorot and Bengio, 2010

Exact solutions to the nonlinear dynamics of learning in deep linear neural networks by Saxe et al, 2013

Random walk initialization for training very deep feedforward networks by Sussillo and Abbott, 2014

Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification by He et al., 2015

Data-dependent Initializations of Convolutional Neural Networks by Krähenbühl et al., 2015

All you need is a good init, Mishkin and Matas, 2015

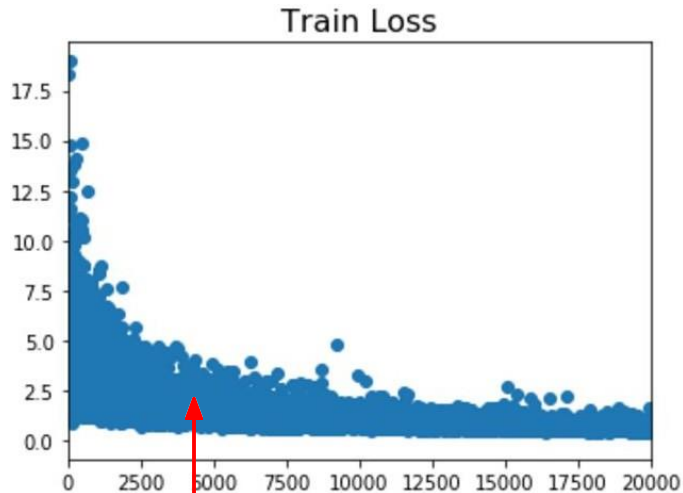
Fixup Initialization: Residual Learning Without Normalization, Zhang et al, 2019

The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks, Frankle and Carbin, 2019

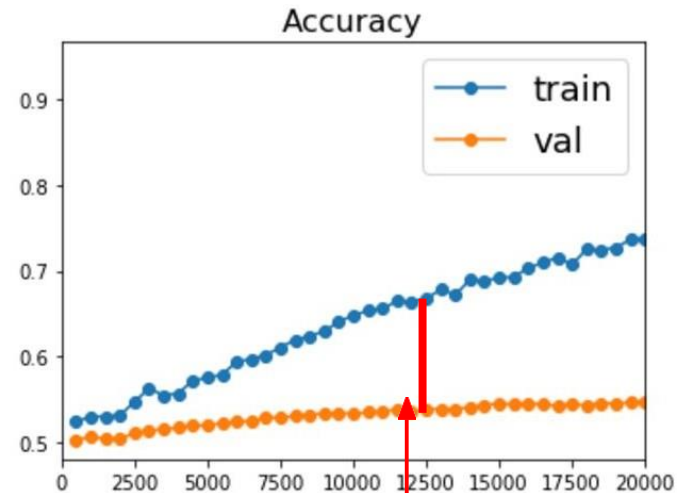
Training vs. Testing Error

Training vs. Testing Error

Beyond Training Error



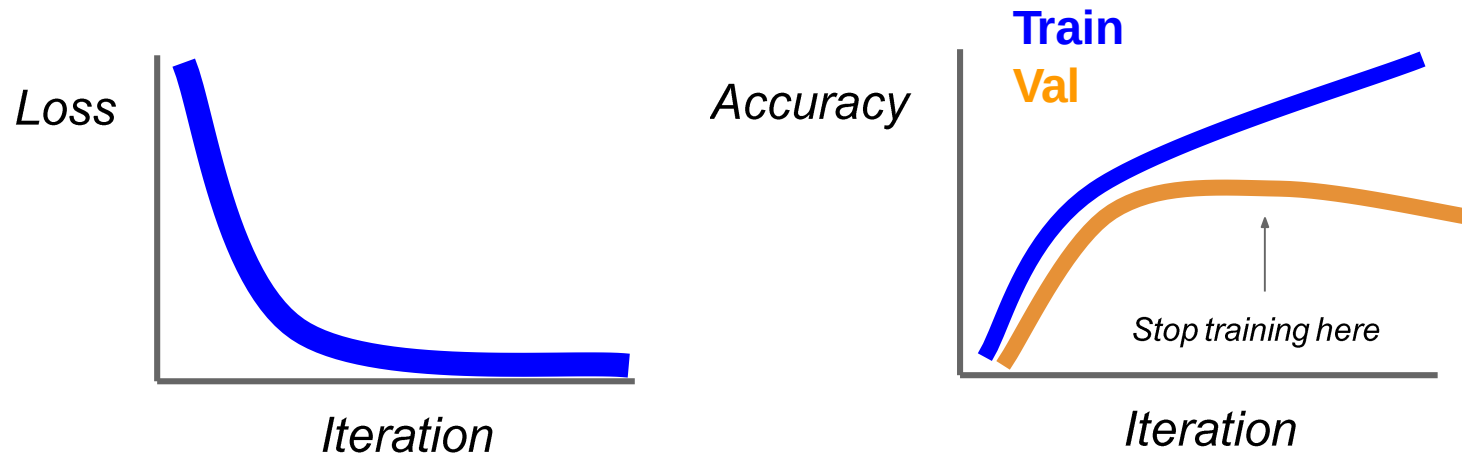
Better optimization algorithms help reduce training loss



But we really care about error on new data - how to reduce the gap?

Training vs. Testing Error

Early Stopping: Always do this



Stop training the model when accuracy on the validation set decreases Or train for a long time, but always keep track of the model snapshot that worked best on val



Training vs. Testing Error

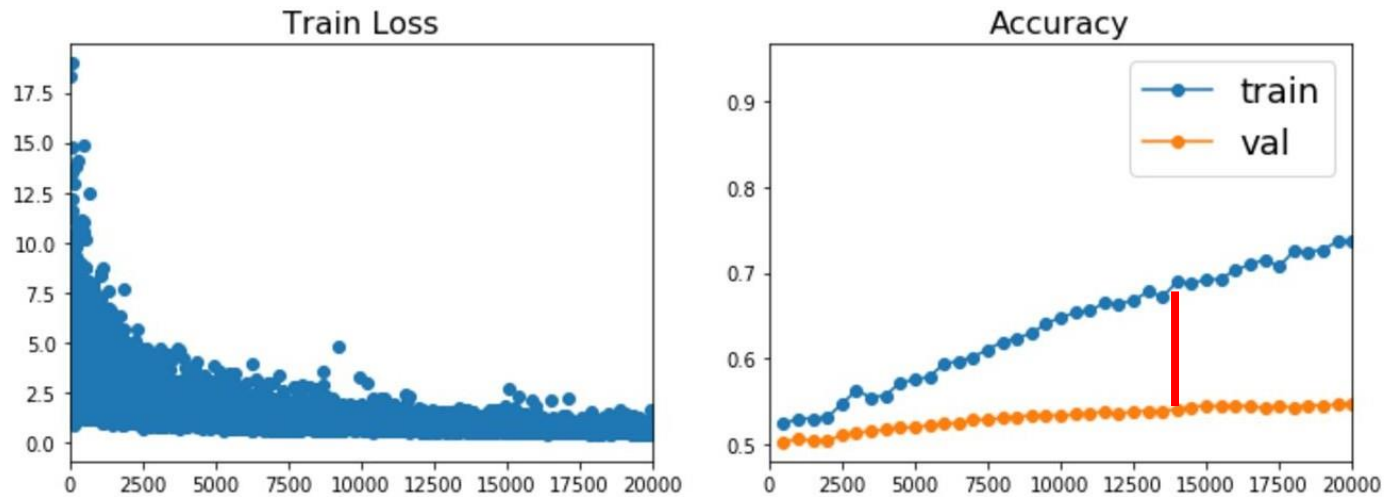
Model Ensembles

1. *Train multiple independent models*
2. *At test time average their results*
(Take average of predicted probability distributions, then choose argmax)

Enjoy 2% extra performance

Training vs. Testing Error

How to improve single-model performance?



Regularization

Training vs. Testing Error

Regularization: Add term to loss

$$L = \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, f(x_i; W)_j - f(x_i; W)_{y_i} + 1) + \lambda R(W)$$

In common use:

L2 regularization

$$R(W) = \sum_k \sum_l W_{k,l}^2 \quad (\text{Weight decay})$$

L1 regularization

$$R(W) = \sum_k \sum_l |W_{k,l}|$$

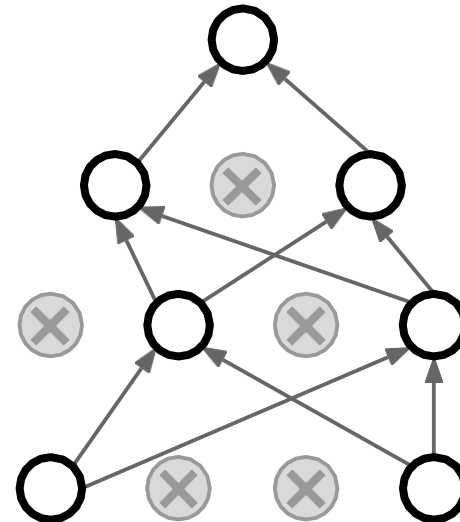
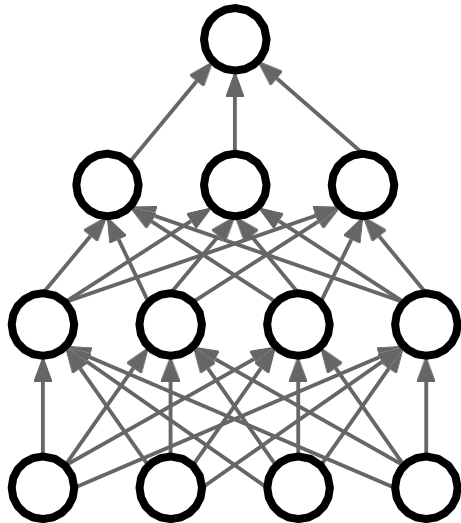
Elastic net (L1 + L2)

$$R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$$

Training vs. Testing Error

Regularization: Dropout

In each forward pass, randomly set some neurons to zero
Probability of dropping is a hyperparameter; 0.5 is common



Srivastava et al, "Dropout: A simple way to prevent neural networks from overfitting", JMLR 2014

Training vs. Testing Error

Regularization: Dropout

```
p = 0.5 # probability of keeping a unit active. higher = less dropout
```

```
def train_step(X):
```

```
    """ X contains the data """
```

```
    # forward pass for example 3-layer neural network
```

```
    H1 = np.maximum(0, np.dot(W1, X) + b1)
```

```
    U1 = np.random.rand(*H1.shape) < p # first dropout mask
```

```
    H1 *= U1 # drop!
```

```
    H2 = np.maximum(0, np.dot(W2, H1) + b2)
```

```
    U2 = np.random.rand(*H2.shape) < p # second dropout mask
```

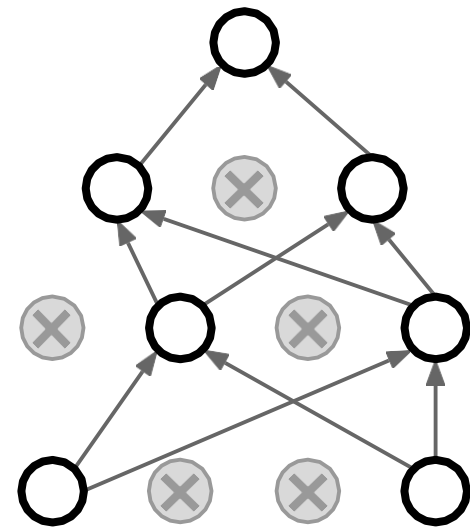
```
    H2 *= U2 # drop!
```

```
    out = np.dot(W3, H2) + b3
```

```
    # backward pass: compute gradients... (not shown)
```

```
    # perform parameter update... (not shown)
```

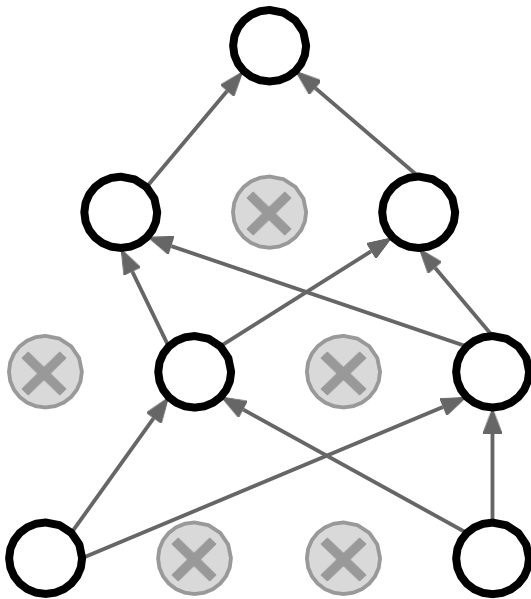
Example forward pass with a 3-layer network using dropout



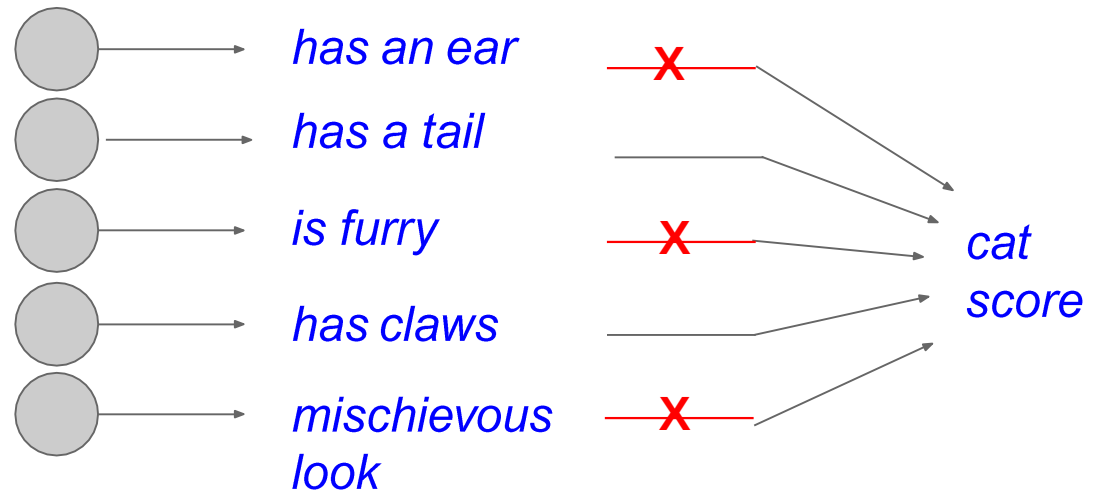
Training vs. Testing Error

Regularization: Dropout

How can this possibly be a good idea?



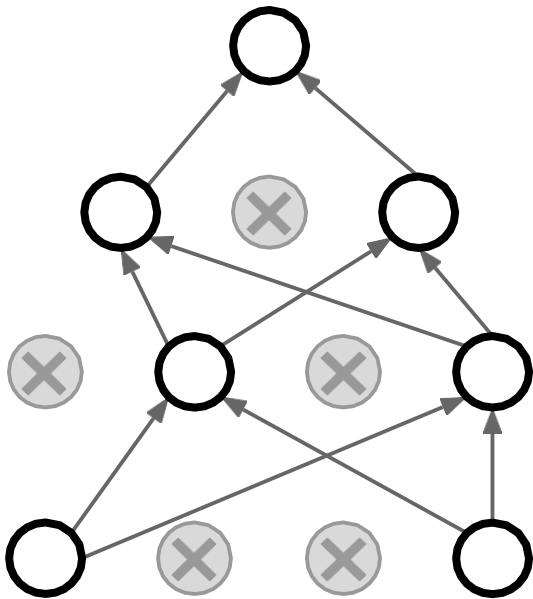
Forces the network to have a redundant representation; Prevents co-adaptation of features



Training vs. Testing Error

Regularization: Dropout

How can this possibly be a good idea?



Another interpretation:

*Dropout is training a large **ensemble** of models (that share parameters).*

Each binary mask is one model

An FC layer with 4096 units has $2^{4096} \sim 10^{1233}$ possible masks!

Only $\sim 10^{82}$ atoms in the universe...

Training vs. Testing Error

Dropout: Test time

Dropout makes our output random!

*Output
(label)*

*Input
(image)*

$$\boxed{y} = f_W(x, \boxed{z})$$

*Random
mask*

Want to “average out” the randomness at test-time

$$y = f(x) = E_z[f(x, z)] = \int p(z) f(x, z) dz$$

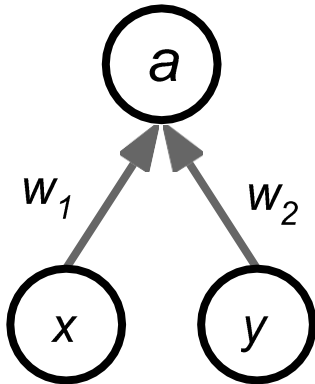
But this integral seems hard ...

Training vs. Testing Error

Dropout: Test time

Want to approximate the integral $y = f(x) = E_z [f(x, z)] = \int p(z) f(x, z) dz$

Consider a single neuron.



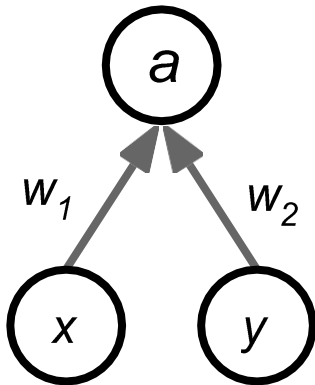
Training vs. Testing Error

Dropout: Test time

Want to approximate the integral $y = f(x) = E_z [f(x, z)] = \int p(z) f(x, z) dz$

Consider a single neuron.

At test time we have: $E[a] = w_1 x + w_2 y$



Training vs. Testing Error

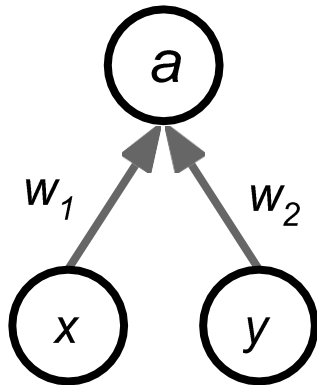
Dropout: Test time

Want to approximate the integral $y = f(x) = E_z [f(x, z)] = \int p(z) f(x, z) dz$

Consider a single neuron.

At test time we have: $E[a] = w_1 x + w_2 y$

During training we have:



$$\begin{aligned} E[a] &= \frac{1}{4}(w_1 x + w_2 y) + \frac{1}{4}(w_1 x + 0y) \\ &\quad + \frac{1}{4}(0x + 0y) + \frac{1}{4}(0x + w_2 y) \\ &= \frac{1}{2}(w_1 x + w_2 y) \end{aligned}$$

Training vs. Testing Error

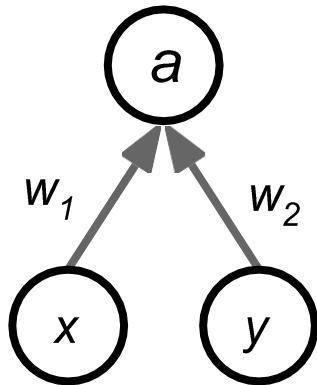
Dropout: Test time

Want to approximate the integral $y = f(x) = E_z [f(x, z)] = \int p(z) f(x, z) dz$

Consider a single neuron.

At test time we have: $E[a] = w_1x + w_2y$

During training we have:



$$\begin{aligned} E[a] &= \frac{1}{4}(w_1x + w_2y) + \frac{1}{4}(w_1x + 0y) \\ &\quad + \frac{1}{4}(0x + 0y) + \frac{1}{4}(0x + w_2y) \\ &= \frac{1}{2}(w_1x + w_2y) \end{aligned}$$

**At test time, multiply
by dropout probability**

Training vs. Testing Error

Dropout: Test time

```
def predict(X):  
    # ensembled forward pass  
    H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the activations  
    H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the activations  
    out = np.dot(W3, H2) + b3
```

At test time all neurons are active always
=> We must scale the activations so that for each neuron:
output at test time = expected output at training time

Training vs. Testing Error

Dropout Summary

```
""" Vanilla Dropout: Not recommended implementation (see notes below) """
```

```
p = 0.5 # probability of keeping a unit active. higher = less dropout
```

```
def train_step(X):
```

```
    """ X contains the data """
```

```
    # forward pass for example 3-layer neural network
```

```
    H1 = np.maximum(0, np.dot(W1, X) + b1)
```

```
    U1 = np.random.rand(*H1.shape) < p # first dropout mask
```

```
    H1 *= U1 # drop!
```

```
    H2 = np.maximum(0, np.dot(W2, H1) + b2)
```

```
    U2 = np.random.rand(*H2.shape) < p # second dropout mask
```

```
    H2 *= U2 # drop!
```

```
    out = np.dot(W3, H2) + b3
```

```
    # backward pass: compute gradients... (not shown)
```

```
    # perform parameter update... (not shown)
```

```
def predict(X):
```

```
    # ensembled forward pass
```

```
    H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the activations
```

```
    H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the activations
```

```
    out = np.dot(W3, H2) + b3
```

drop in train time

scale at test time

Training vs. Testing Error

More common: “Inverted dropout”

```
p = 0.5 # probability of keeping a unit active. higher = less dropout

def train_step(X):
    # forward pass for example 3-layer neural network
    H1 = np.maximum(0, np.dot(W1, X) + b1)
    U1 = (np.random.rand(*H1.shape) < p) / p # first dropout mask. Notice /p!
    H1 *= U1 # drop!
    H2 = np.maximum(0, np.dot(W2, H1) + b2)
    U2 = (np.random.rand(*H2.shape) < p) / p # second dropout mask. Notice /p!
    H2 *= U2 # drop!
    out = np.dot(W3, H2) + b3

    # backward pass: compute gradients... (not shown)
    # perform parameter update... (not shown)

def predict(X):
    # ensembled forward pass
    H1 = np.maximum(0, np.dot(W1, X) + b1) # no scaling necessary
    H2 = np.maximum(0, np.dot(W2, H1) + b2)
    out = np.dot(W3, H2) + b3
```

test time is unchanged!

Training vs. Testing Error

Regularization: A common pattern

Training: *Add some kind of randomness*

$$y = f_W(x, z)$$

Testing: *Average out randomness (sometimes approximate)*

$$y = f(x) = E_z[f(x, z)] = \int p(z)f(x, z)dz$$



Training vs. Testing Error

Regularization: A common pattern

Training: *Add some kind of randomness*

$$y = f_W(x, z)$$

Testing: *Average out randomness
(sometimes approximate)*

$$y = f(x) = E_z[f(x, z)] = \int p(z) f(x, z) dz$$

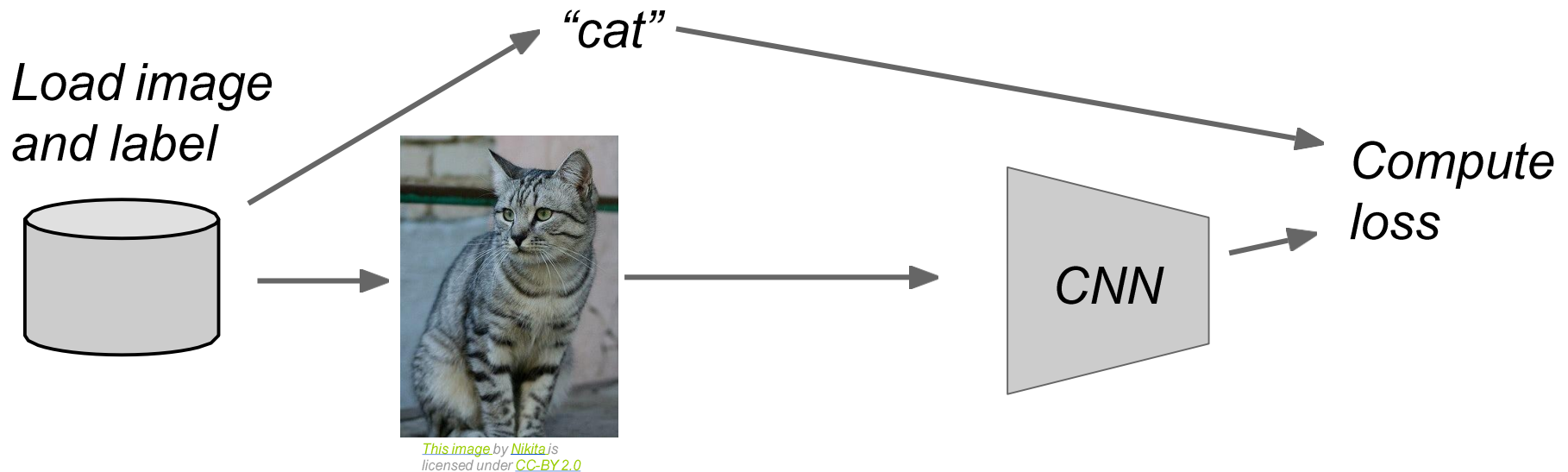
Example: *Batch Normalization*

Training:
*Normalize using
stats from random
minibatches*

Testing: *Use fixed
stats to normalize*

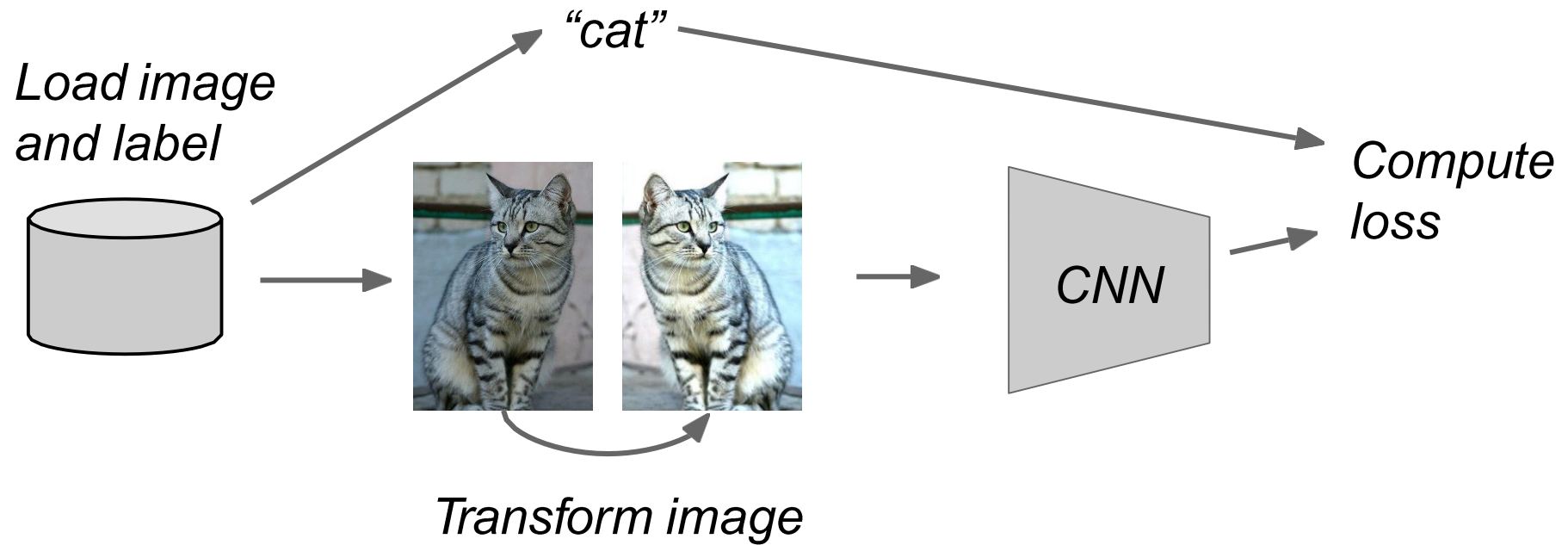
Training vs. Testing Error

Regularization: Data Augmentation



Training vs. Testing Error

Regularization: Data Augmentation



Training vs. Testing Error

Data Augmentation Horizontal Flips



Training vs. Testing Error

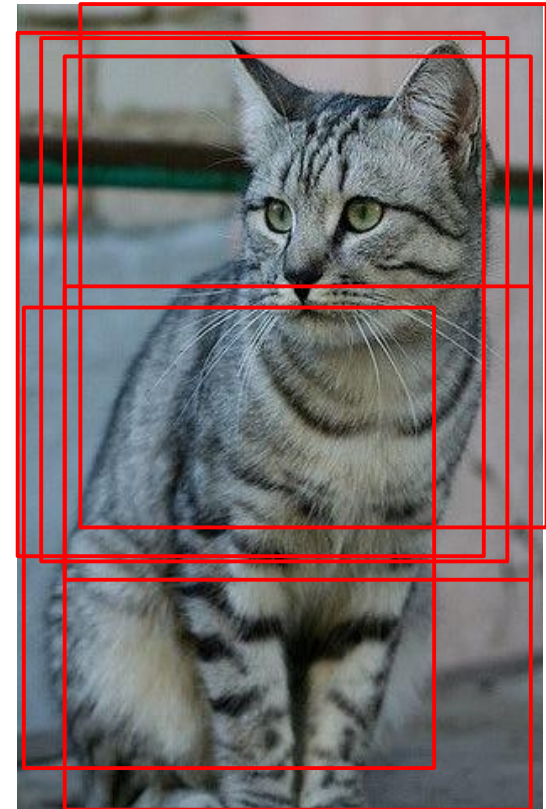
Data Augmentation

Random crops and scales

Training: *sample random crops / scales*

ResNet:

1. *Pick random L in range $[256, 480]$*
2. *Resize training image, short side = L*
3. *Sample random 224×224 patch*



Training vs. Testing Error

Data Augmentation

Random crops and scales

Training: *sample random crops / scales*

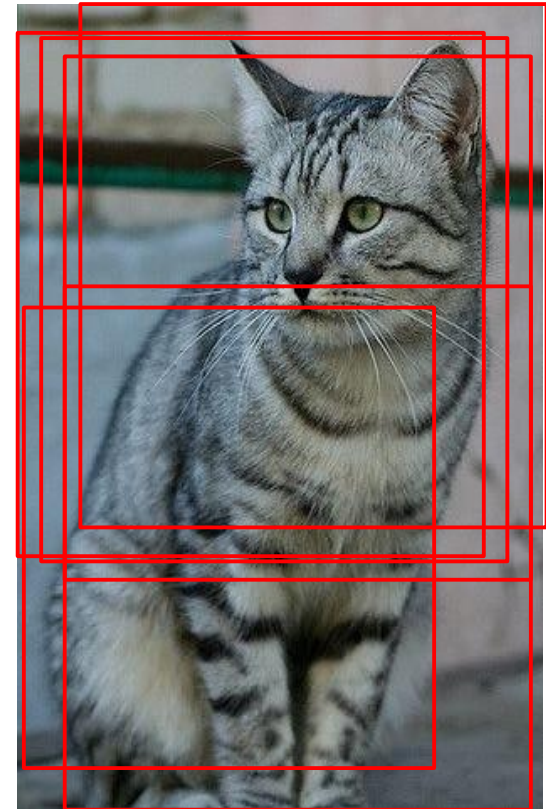
ResNet:

1. *Pick random L in range $[256, 480]$*
2. *Resize training image, short side = L*
3. *Sample random 224×224 patch*

Testing: *average a fixed set of crops*

ResNet:

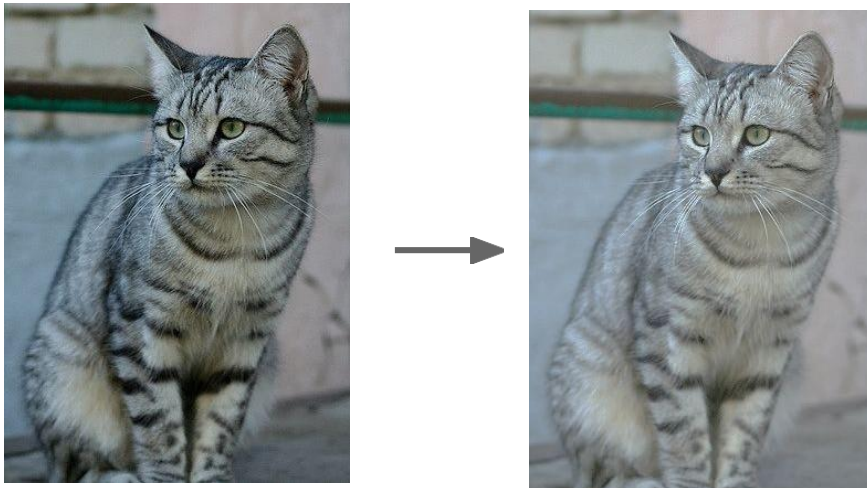
1. *Resize image at 5 scales:
 $\{224, 256, 384, 480, 640\}$*
2. *For each size, use 10 224×224 crops:
4 corners + center, + flips*



Training vs. Testing Error

Data Augmentation
Color Jitter

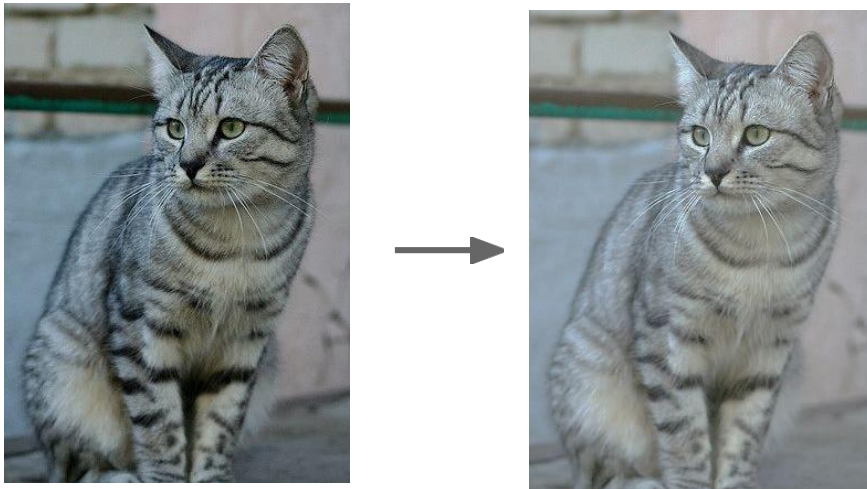
Simple: *Randomize
contrast and brightness*



Training vs. Testing Error

Data Augmentation Color Jitter

Simple: *Randomize contrast and brightness*



More Complex:

1. *Apply PCA to all $[R, G, B]$ pixels in training set*
2. *Sample a “color offset” along principal component directions*
3. *Add offset to all pixels of a training image*

(As seen in [Krizhevsky et al. 2012], ResNet, etc)

Training vs. Testing Error

Data Augmentation

- **Get creative for your problem!**

Examples of data augmentations:

- *translation*
- *rotation*
- *stretching*
- *shearing,*
- *lens distortions, ...* (go crazy)

Training vs. Testing Error

Automatic Data Augmentation

	Original	Sub-policy 1	Sub-policy 2	Sub-policy 3	Sub-policy 4	Sub-policy 5
Batch 1						
Batch 2						
Batch 3						
		ShearX, 0.9, 7 Invert, 0.2, 3	ShearY, 0.7, 6 Solarize, 0.4, 8	ShearX, 0.9, 4 AutoContrast, 0.8, 3	Invert, 0.9, 3 Equalize, 0.6, 3	ShearY, 0.8, 5 AutoContrast, 0.7, 3

Cubuk et al., “AutoAugment: Learning Augmentation Strategies from Data”, CVPR 2019



Training vs. Testing Error

Regularization: A common pattern

Training: *Add random noise*

Testing: *Marginalize over the noise*

Examples:

Dropout

Batch Normalization

Data Augmentation

Training vs. Testing Error

Regularization: A common pattern

Training: *Drop connections between neurons (set weights to 0)*

Testing: *Use all the connections*

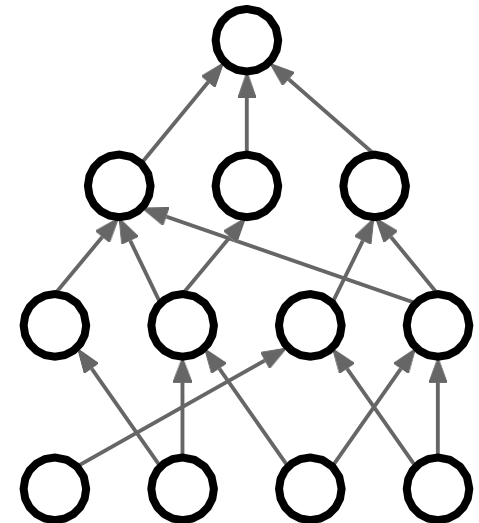
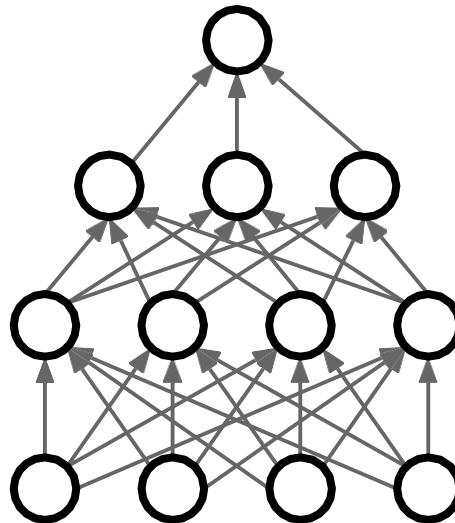
Examples:

Dropout

Batch Normalization

Data Augmentation

Drop Connect



*Wan et al, "Regularization of Neural Networks using DropConnect",
ICML 2013*

Training vs. Testing Error

Regularization: A common pattern

Training: *Use randomized pooling regions*

Testing: *Average predictions from several regions*

Examples:

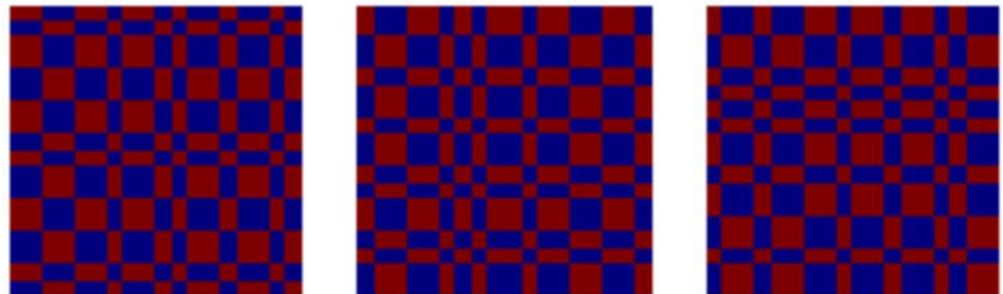
Dropout

Batch Normalization

Data Augmentation

Drop Connect

Fractional Max Pooling



Graham, “Fractional Max Pooling”, arXiv 2014

Training vs. Testing Error

Regularization: A common pattern

Training: Skip some layers in the network

Testing: Use all the layer

Examples:

Dropout

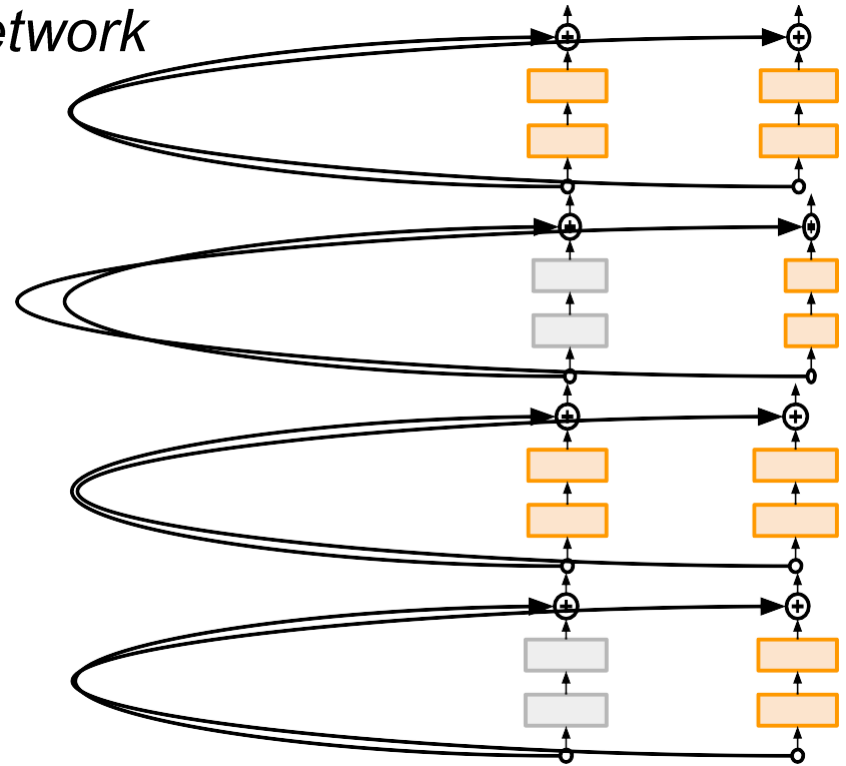
Batch Normalization

Data Augmentation

Drop Connect

Fractional Max Pooling

Stochastic Depth



Huang et al, "Deep Networks with Stochastic Depth", ECCV 2016

Training vs. Testing Error

Regularization: A common pattern

Training: Set random image regions to 0

Testing: Use full image

Examples:

Dropout

Batch Normalization

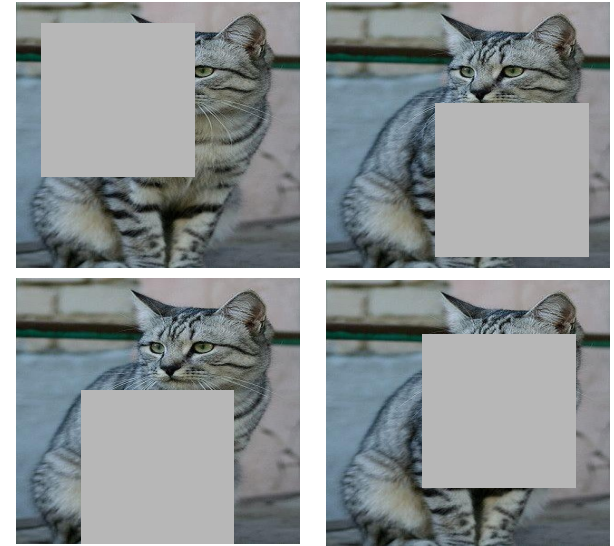
Data Augmentation

Drop Connect

Fractional Max Pooling

Stochastic Depth

Cutout / Random Crop



Works very well for small datasets like CIFAR, less common for large datasets like ImageNet

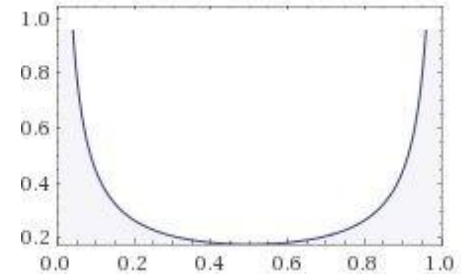
DeVries and Taylor, “Improved Regularization of Convolutional Neural Networks with Cutout” , arXiv 2017

Training vs. Testing Error

Regularization: A common pattern

Training: *Train on random blends of images*

Testing: *Use original images*



Examples:

Dropout

Batch Normalization

Data Augmentation

Drop Connect

Fractional Max Pooling

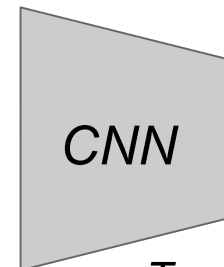
Stochastic Depth

Cutout / Random Crop

Mixup



Randomly blend the pixels of pairs of training images, e.g. 40% cat, 60% dog



Target label:

cat: 0.4

dog: 0.6

Zhang et al, “mixup: Beyond Empirical Risk Minimization”, ICLR 2018

Training vs. Testing Error

Regularization: In practice

Training: *Train on random blends of images*

Testing: *Use original images*

Examples:

Dropout

Batch Normalization

Data Augmentation

Drop Connect

Fractional Max Pooling

Stochastic Depth

Cutout / Random Crop

Mixup

- *Consider dropout for large fully-connected layers*
- *Batch normalization and data augmentation almost always a good idea*
- *Try cutout and mixup especially for small classification datasets*

Choosing Hyperparameters

(without tons of GPUs)



Choosing Hyperparameters

Step 1: *Check initial loss*

*Turn off weight decay, sanity check loss at initialization
e.g. $\log(C)$ for softmax with C classes*

Random guessing $\rightarrow 1/C$ probability for each class
Softmax Loss $\rightarrow -\log(1/C) = \log(C)$



Choosing Hyperparameters

Step 1: *Check initial loss*

Step 2: *Overfit a small sample*

Try to train to 100% training accuracy on a small sample of training data (~5-10 minibatches); fiddle with architecture, learning rate, weight initialization

Loss not going down? LR too low, bad initialization

Loss explodes to Inf or NaN? LR too high, bad initialization



Choosing Hyperparameters

Step 1: *Check initial loss*

Step 2: *Overfit a small sample*

Step 3: *Find LR that makes loss go down*

Use the architecture from the previous step, use all training data, turn on small weight decay, find a learning rate that makes the loss drop significantly within ~100 iterations

Good learning rates to try: $1e-1$, $1e-2$, $1e-3$, $1e-4$



Choosing Hyperparameters

Step 1: *Check initial loss*

Step 2: *Overfit a small sample*

Step 3: *Find LR that makes loss go down*

Step 4: *Coarse grid, train for ~1-5 epochs*

Choose a few values of learning rate and weight decay around what worked from Step 3, train a few models for ~1-5 epochs.

Good weight decay to try: $1e-4$, $1e-5$, 0



Choosing Hyperparameters

Step 1: *Check initial loss*

Step 2: *Overfit a small sample*

Step 3: *Find LR that makes loss go down*

Step 4: *Coarse grid, train for ~1-5 epochs*

Step 5: *Refine grid, train longer*

Pick best models from Step 4, train them for longer (~10-20 epochs) without learning rate decay



Choosing Hyperparameters

Step 1: *Check initial loss*

Step 2: *Overfit a small sample*

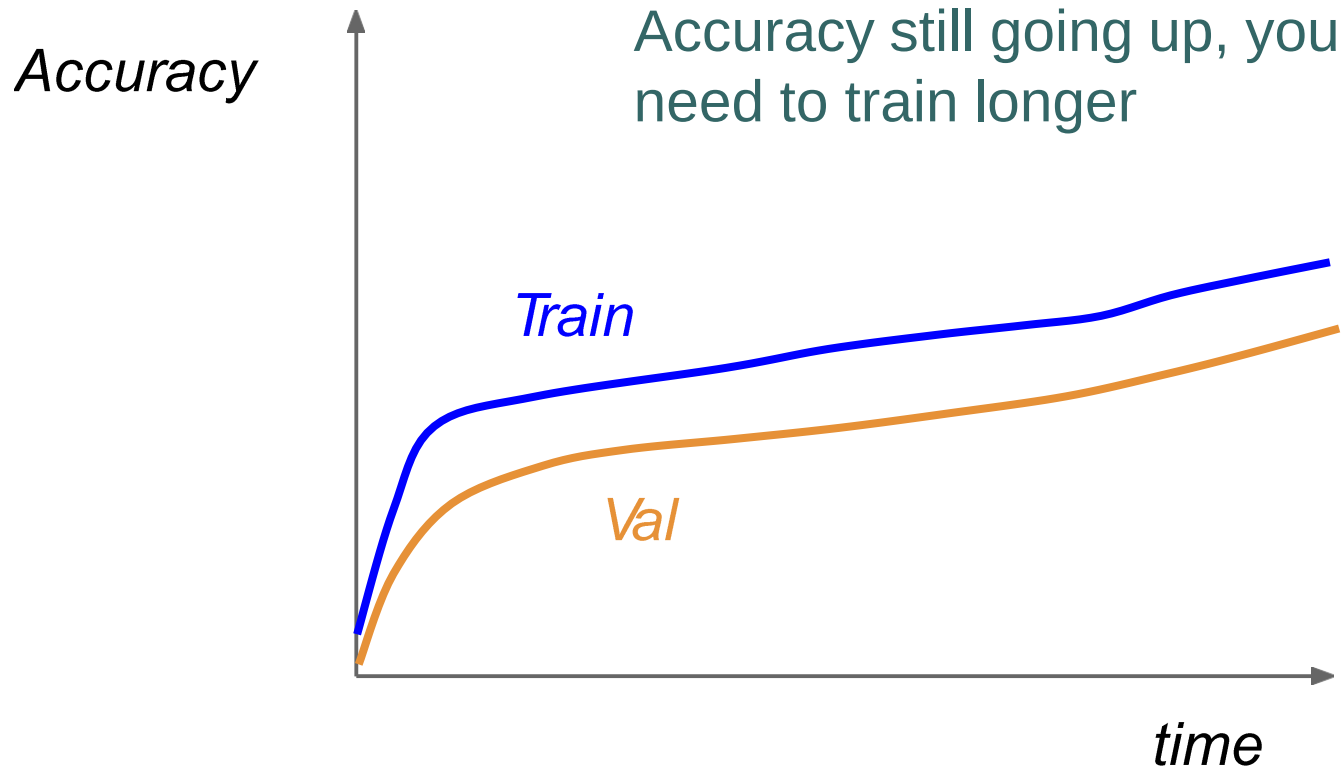
Step 3: *Find LR that makes loss go down*

Step 4: *Coarse grid, train for ~1-5 epochs*

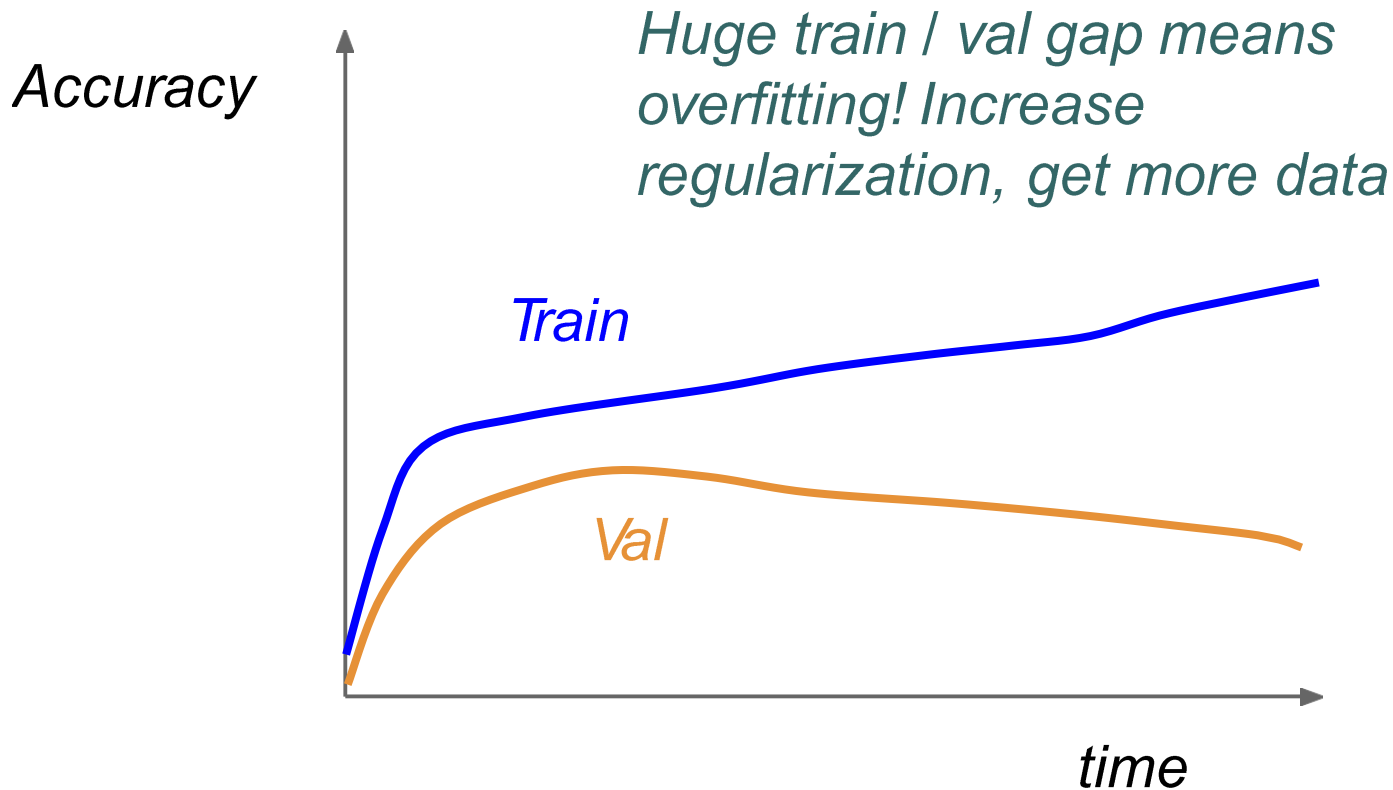
Step 5: *Refine grid, train longer*

Step 6: *Look at loss and accuracy curves*

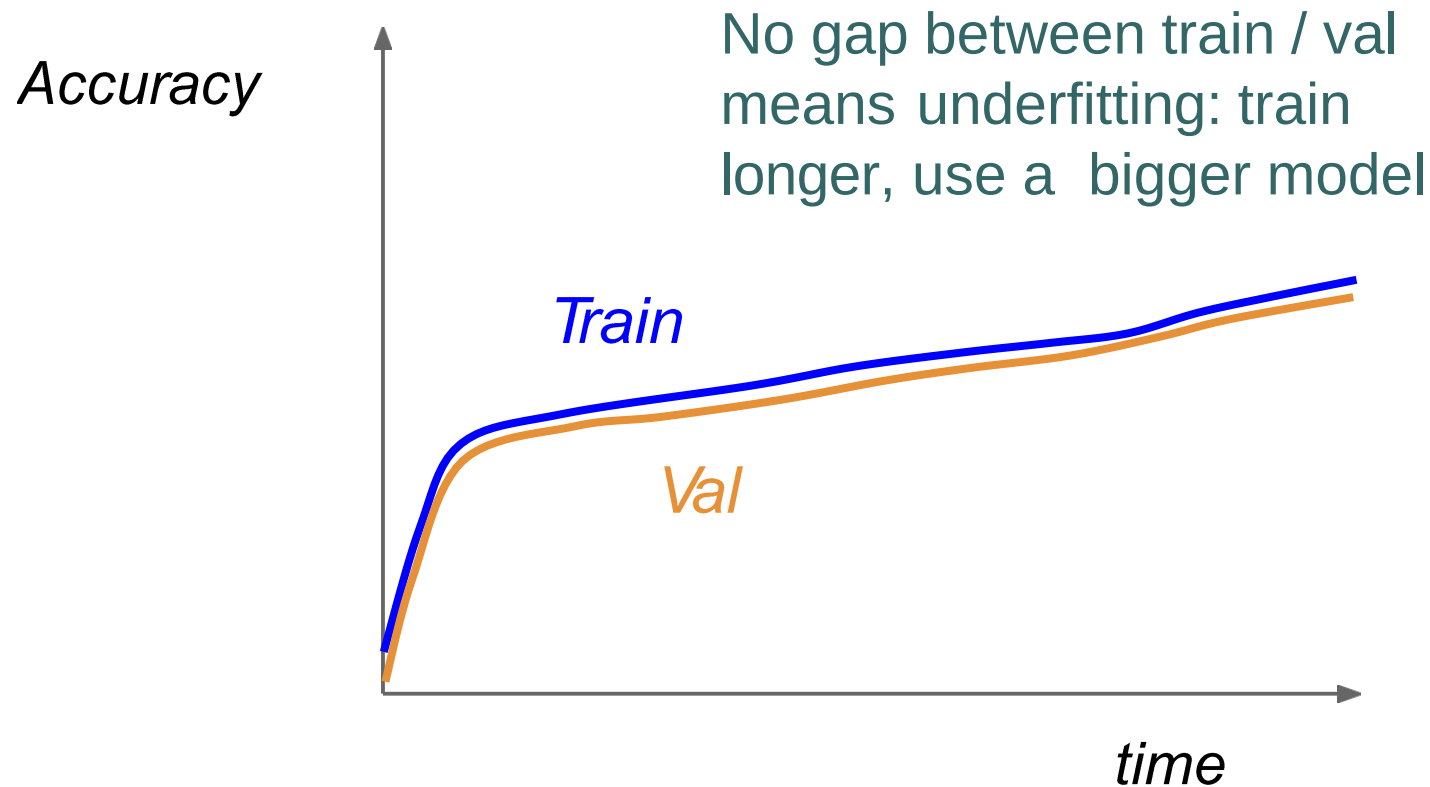
Choosing Hyperparameters



Choosing Hyperparameters



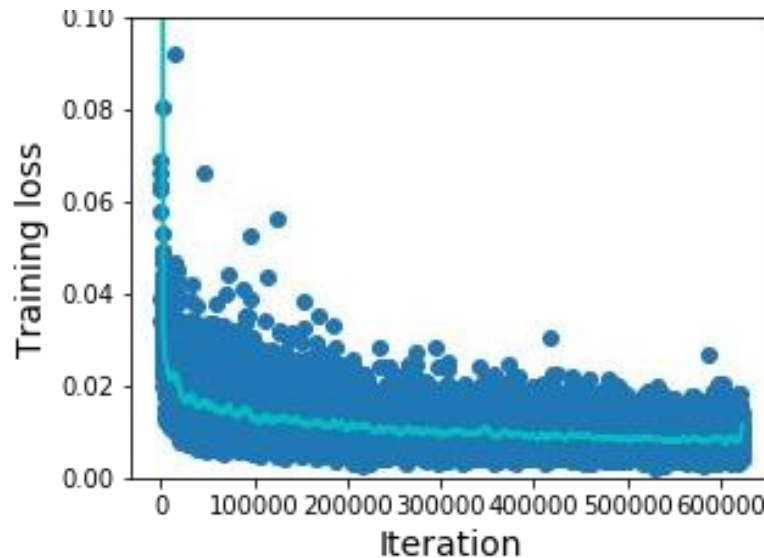
Choosing Hyperparameters



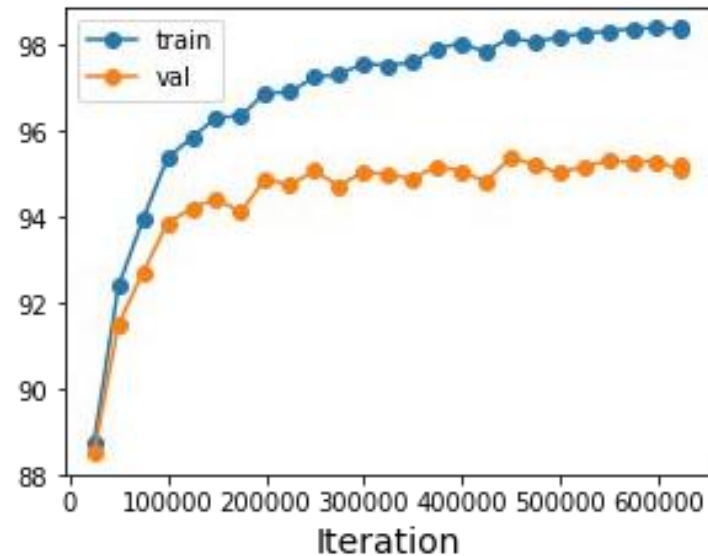
Choosing Hyperparameters

Look at learning curves!

Training Loss



Train / Val Accuracy

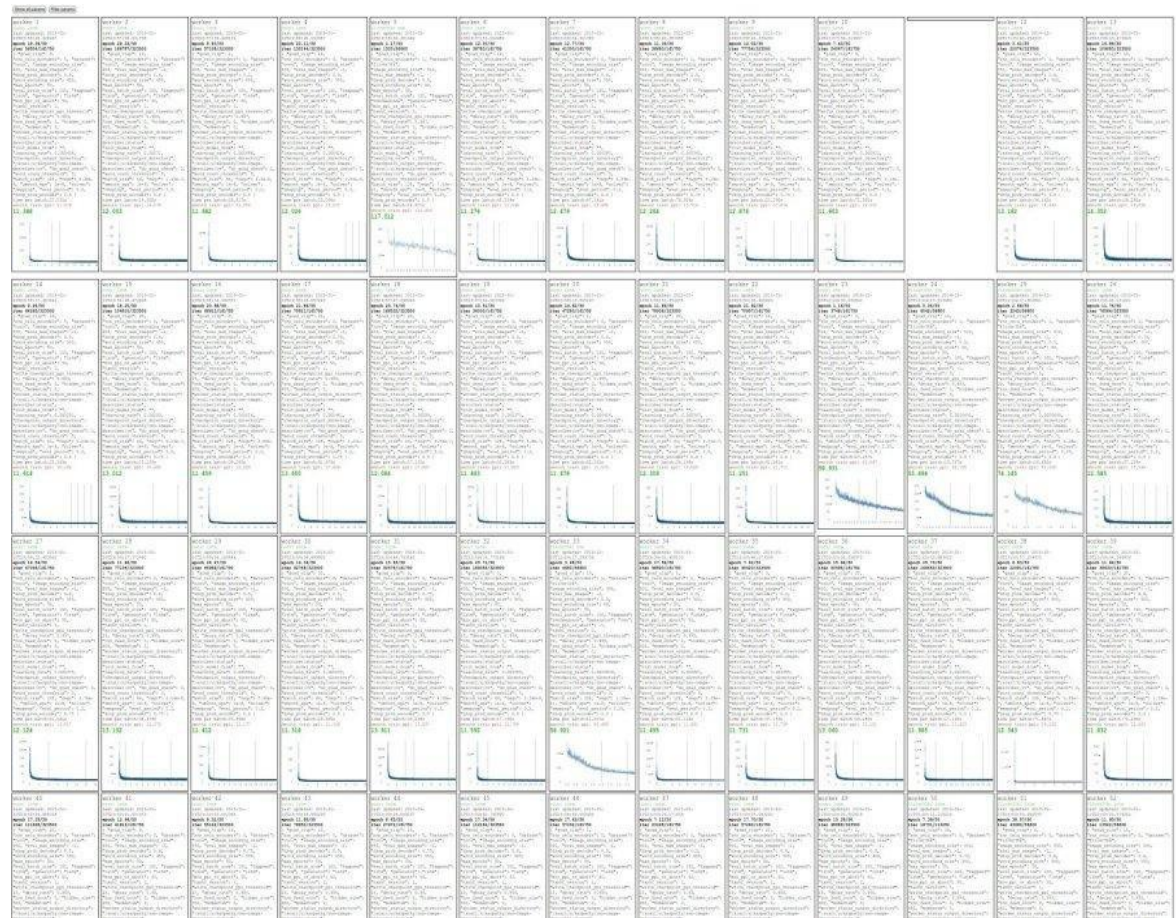


Losses may be noisy, use a scatter plot and also plot moving average to see trends better

Choosing Hyperparameters

Cross-validation

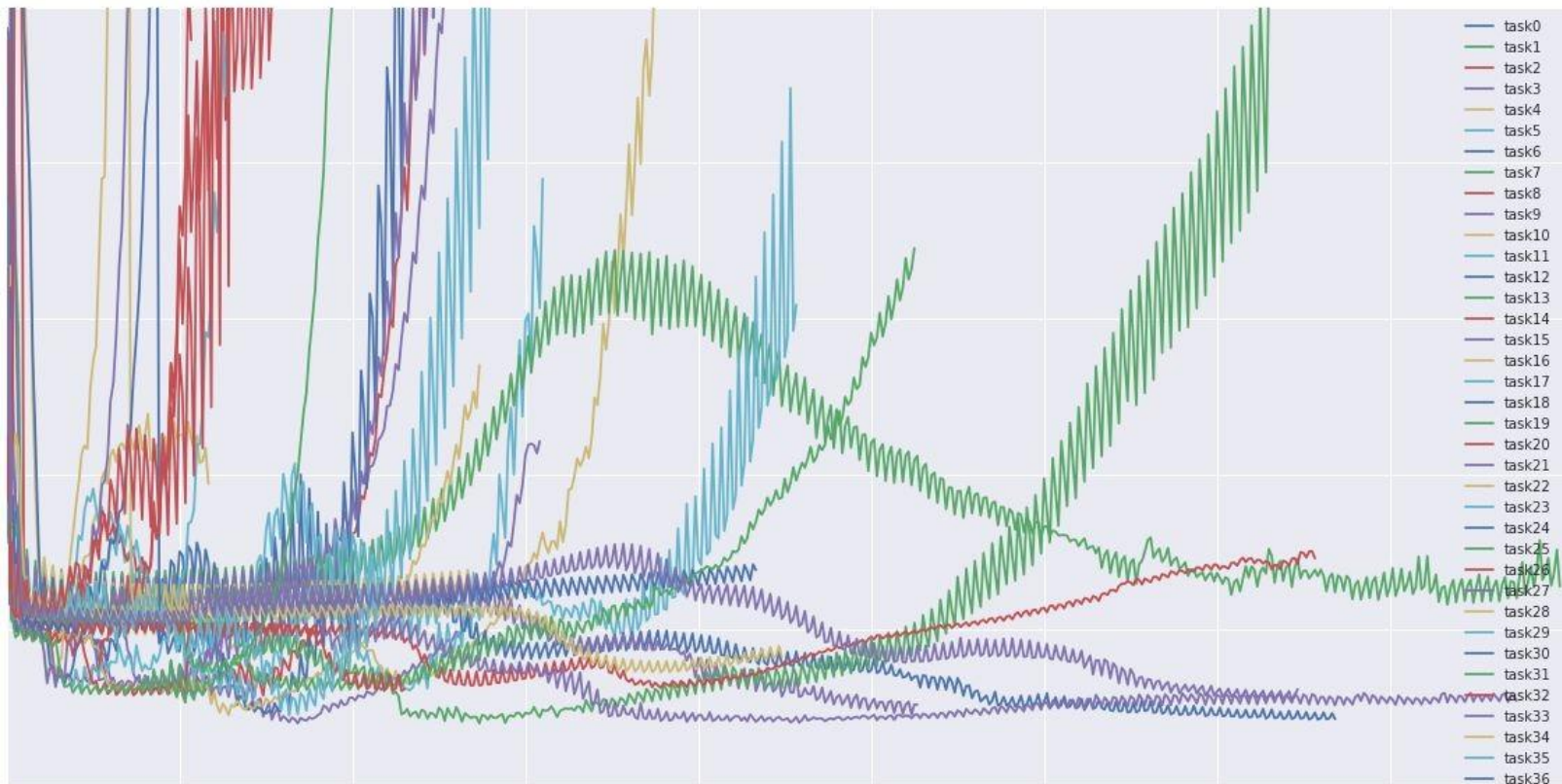
We develop "command centers" to visualize all our models training with different hyperparameters



check out [weights and biases](#)

Choosing Hyperparameters

You can plot all your loss curves for different hyperparameters on a single plot



Choosing Hyperparameters

Don't look at accuracy or loss curves for too long!





Choosing Hyperparameters

Step 1: *Check initial loss*

Step 2: *Overfit a small sample*

Step 3: *Find LR that makes loss go down*

Step 4: *Coarse grid, train for ~1-5 epochs*

Step 5: *Refine grid, train longer*

Step 6: *Look at loss and accuracy curves*

Step 7: ***GOTO step 5***

Choosing Hyperparameters

Random Search vs. Grid Search

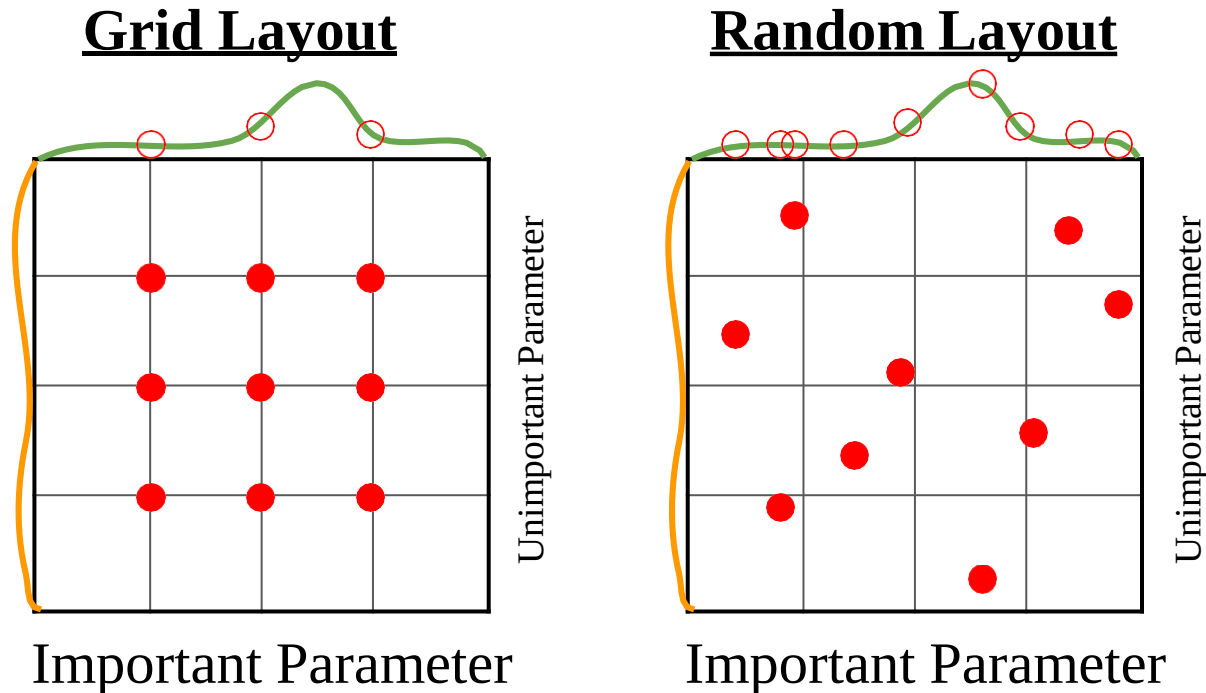


Illustration of Bergstra et al., 2012 by Shayne Longpre, copyright CS231n 2017

Random Search for Hyper-Parameter Optimization
Bergstra and Bengio, 2012

Summary

- **Improve your training error:**
 - *Optimizers*
 - *Learning rate schedules*
- **Improve your test error:**
 - *Regularization*
 - *Choosing Hyperparameters*



Summary

- **We looked in detail at:**

TLDRs

- *Activation Functions (use ReLU)*
- *Data Preprocessing (images: subtract mean)*
- *Weight Initialization (use Xavier/He init)*
- *Batch Normalization (use this!)*
- *Transfer learning (use this if you can!)*